

Auraly ■ — Technical Blueprint Manual

Detailed System Architecture, Subsystem Menus & Operating Reference

Document Version: 1.1 (Full Specification with Context Menu)	Classification: Technical Reference
Target Operating System: Windows 10 & Windows 11 (x64)	Core GUI Engine: PyQt6 Framework
Media Session Hook: WinRT GSMTc API	Lyrics Provider: LRCLIB REST Service
Packaging: PyInstaller Standalone Executable	Repository License: MIT Open Source

1. Executive Technical Briefing & Problem Space Analysis

Auraly is an ultra-lightweight desktop lyrics synchronization overlay engineered specifically for the Microsoft Windows operating system. Designed to integrate directly with Windows 10 (Build 19041+) and Windows 11 taskbars, Auraly delivers real-time synchronized music lyrics without cluttering screen real estate or interrupting workflow.

Solution Type	Screen Footprint	Resource Impact	Synchronization Precision
Auraly (Taskbar Overlay)	Zero (Taskbar anchored at x=12)	< 25 MB RAM / < 0.1% CPU	Sub-second UTC Offset Drift Math
Full Desktop Apps	High (Requires open window)	150 - 400 MB RAM	Variable (App dependent)
Browser Extensions	Medium (Bound to browser tab)	Heavy Chromium process load	Browser tab locked only
On-Screen Widgets	High (Floats over windows)	50 - 120 MB RAM	Polling based lag

2. Context Menu Subsystem & Feature Set (`\_show\_context\_menu\_at\_pos`)

Right-clicking anywhere on the Auraly taskbar overlay or system tray icon triggers a clean, sleek, context-aware popup menu. Designed without repetitive icon clutter, the context menu provides instant control over positioning, typography, transparency, and manual synchronization.

Context Menu Feature	Underlying Code Action & Functionality	User & Operational Impact
Refresh / Resync Lyrics	Emits `refresh_requested_signal`. Flushes active lyric cache and forces an immediate HTTP GET re-query to LRCLIB API.	Allows manual recovery if media metadata was briefly out of sync or updated mid-song.
Lock / Unlock Position	Toggles `self.is_locked`. When unlocked, renders a green dashed bounding box allowing left-click dragging anywhere on screen.	Provides drag-and-drop relocation across multi-monitor setups or custom desktop layouts.
Click-Through Mode	Toggles Win32 window extended styles: `WS_EX_TRANSPARENT` & `WS_EX_NOACTIVATE` via `SetWindowLongW`.	Makes the overlay 100% click-transparent, allowing mouse clicks to pass straight through to background windows.
Reset to Left Taskbar	Calls `position_on_left_taskbar()`. Re-queries `Shell_TrayWnd` taskbar rect and snaps overlay back to `x=12`.	Instantly restores default ergonomic taskbar docking if window was dragged away.

Opacity Preset Submenu	Dynamically updates `self.opacity_percent` across 5 presets: 10%, 25%, 50%, 80%, 100% .	Adjusts text alpha transparency for optimal legibility against dark or light wallpapers.
Font Family Submenu	Switches `self.font_family` to selected geometric sans-serif font: Century Gothic, Montserrat, Poppins, Outfit, Segoe UI, Arial .	Instant real-time typography customization without restarting the application.
Change Font Size	Invokes PyQt6 `QInputDialog.getInt()`, allowing font size selection between 8pt and 24pt .	Enables font size scaling for high-DPI 4K displays or compact 1080p monitors.

3. Detailed Multi-Threaded Process Architecture

Auraly employs an event-driven, multi-threaded architecture that isolates heavy tasks—such as Windows media session polling and HTTP REST requests—from the PyQt6 main user interface thread. This prevents frame drops, window freezing, or UI stuttering.

Thread Context	Primary Responsibility	Communication Mechanism
Main Thread (PyQt6 UI)	Executes `QApplication` event loop, manages Win32 taskbar positioning, renders transparent window overlay, updates lyric text.	Receives Qt Signals (`media_changed`, `position_changed`, `lyrics_ready`).
Media Listener Thread (`QThread`)	Runs an `asyncio` event loop. Hooks into WinRT `GlobalSystemMediaTransportControlsSessionManager` to poll active playback metadata.	Emits `pyqtSignal(str, str, str)` on track change & `pyqtSignal(int)` on position pulse.
Lyrics Fetcher Worker	Executes asynchronous REST HTTP GET calls to LRCLIB API (`/api/get` & `/api/search`). Performs query string cleaning.	Emits `lyrics_loaded(dict)` signal back to Main Thread.

4. Minute Technical Execution Flow (Subsystem Level)

Below is an exhaustive step-by-step breakdown of every minute operation that executes within Auraly from launch to real-time lyric sync rendering.

Phase 1: Boot & Single-Instance Socket Locking
- main.py invokes <code>socket.socket(socket.AF_INET, socket.SOCK_STREAM)</code>. - Attempts to bind to <code>127.0.0.1:54321</code>. - If another Auraly process is running, <code>bind()</code> raises <code>OSError (WSAEADDRINUSE)</code>. Auraly prints a duplicate instance log and immediately calls <code>sys.exit(0)</code>. - If successful, the socket handle is retained in memory for the process lifetime.
Phase 2: WinRT Media Control Hooking & Session Discovery
- media_listener.py initializes <code>winrt.windows.media.control.GlobalSystemMediaTransportControlsSessionManager</code>. - Calls <code>request_async()</code> to get current active media sessions. - Attaches session change listener: <code>session_manager.add_current_session_changed(on_session_changed)</code>. - Works seamlessly with Spotify, Chrome, Edge, Apple Music, VLC, Windows Media Player, and Tidal.
Phase 3: High-Precision UTC Position & Drift Mathematics
- WinRT media timeline properties only update periodically (e.g. every few seconds). - To achieve smooth sub-second line switching, Auraly calculates live position using system UTC ticks: <code>live_position_ms = reported_position_ms + (current_utc_ticks - last_update_utc_ticks) * playback_rate</code> - When media is paused, drift addition is suspended (<code>rate = 0</code>). When track seeks occur, position updates instantly.
Phase 4: LRCLIB API Query & Normalization Pipeline

- On song change, lyrics_fetcher.py receives (title, artist, album).
- Query strings undergo regex cleaning:
 - Strips parenthetical tags: `r'\(.*?\)', r'\[.*?\]'` (e.g., '(Remastered 2020)', '[Deluxe]').
 - Strips feature tags: `r'(?i)feat\..*'`.
- Issues HTTP GET: `https://lrclib.net/api/get?track_name=...&artist_name=....`
- If exact match fails (404), issues fallback search: `/api/search?q=artist+track`.

Phase 5: LRC Timestamp Parsing & Array Construction

- lrc_parser.py parses raw ``lrc`` text payloads.
- Matches timestamps using regex: `\[(\d{2}):(\d{2})\.(\d{2,3})\](.*)`.
- Converts time components: `ms = (minutes * 60 + seconds) * 1000 + (fraction * 10)`.
- Constructs sorted tuple array: `[(timestamp_ms, lyric_text), ...]`.
- Implements $O(\log N)$ binary search lookup: `get_line_at_ms(current_position_ms)`.

Phase 6: Win32 Taskbar Geometry Anchoring

- taskbar_overlay.py calls Win32 User32 API: `FindWindowW('Shell_TrayWnd', None)`.
- Retrieves taskbar bounding box via `GetWindowRect(hwnd, ■)`.
- Computes dynamic placement coordinates:
 - `x = 12` (Anchored left near Start button)
 - `y = rect.top - overlay_height - 6` (Docked 6px above taskbar top edge).
- Window stays visible above taskbar without stealing focus (``Qt.WindowType.Tool``).

Phase 7: Century Gothic Rendering & Smooth Transitions

- Renders lyrics using **Century Gothic** geometric sans-serif typography at 14pt semi-bold.
- Configures 80% window opacity (``setWindowOpacity(0.80)``).
- Implements smooth line transitions: when line changes, updates ``QLabel`` text with subtle anti-aliased font rendering.

5. Comprehensive Source Code & Module Analysis

This section documents the exact purpose, classes, methods, and line-by-line responsibilities of every file in the Auraly codebase.

main.py — Main Application Controller

- Creates `QApplication(sys.argv)` with high-DPI scaling.
- Implements `socket_lock()` binding `127.0.0.1:54321` to block duplicate processes.
- Instantiates `TaskbarOverlay`, `MediaListener`, `LyricsFetcher`.
- Connects Qt Signals: `media_listener.media_changed -> lyrics_fetcher.on_media_changed`.
- Connects Qt Signals: `media_listener.position_changed -> taskbar_overlay.on_position_changed`.

media_listener.py — WinRT Media Subsystem

- Imports `winrt.windows.media.control.GlobalSystemMediaTransportControlsSessionManager`.
- Runs inside `QThread` with dedicated `asyncio` event loop.
- Polls playback status (PLAYING, PAUSED, STOPPED) every 100 milliseconds.
- Extracts `Title`, `Artist`, `AlbumTitle`, `Position`, `LastUpdatedTime`, `PlaybackRate`.

lyrics_fetcher.py — LRCLIB REST Client

- Imports `requests` and implements REST HTTP client for `lrclib.net`.
- Sanitizes query strings: removes brackets, remaster suffixes, feat tags.
- Issues GET to `/api/get?track_name=...&artist_name=...`.
- Fallback GET to `/api/search?q=...` if exact match fails.
- Parses JSON response containing `syncedLyrics` or `plainLyrics`.

lrc_parser.py — LRC Parser & Synchronization Engine

- Parses LRC text payloads using regex `\[(\d+):(\d+\.\d+)\](.*)`.
- Converts timestamps to milliseconds dictionary.
- Implements `get_current_line(current_ms)` returning `(current_text, next_text)`.
- Supports plain text autoscroll algorithm for unsynchronized songs.

taskbar_overlay.py — PyQt6 Frameless Overlay Window & Context Menu

- Subclasses `QWidget` with flags `FramelessWindowHint | WindowStaysOnTopHint | Tool`.
- Sets background mask to `Qt.WidgetAttribute.WA_TranslucentBackground`.
- Implements right-click context menu `_show_context_menu_at_pos()`.
- Includes Refresh, Lock Position, Click-Through, Taskbar Reset, Opacity, Font Family, Font Size.
- Queries Win32 `FindWindowW('Shell_TrayWnd', None)` to get taskbar rectangle bounds.
- Calculates dynamic screen position at `x=12`, `y=taskbar_top - window_height - 6`.

create_shortcuts.py — Windows Deployment Installer

- Locates Windows Startup folder (`AppData\...\Start Menu\Programs\Startup`).
- Creates Desktop & Startup `.lnk` shortcuts using temporary Windows Script Host VBS script.
- Removes temporary VBS script after shortcut creation.

6. Critical Technical Code Implementations

A. Win32 Taskbar Position Calculation (`taskbar\_overlay.py`)

```
import ctypes
from ctypes import wintypes

class RECT(ctypes.Structure):
    _fields_ = [('left', wintypes.LONG), ('top', wintypes.LONG),
                ('right', wintypes.LONG), ('bottom', wintypes.LONG)]

def update_taskbar_position(self):
    hwnd = ctypes.windll.user32.FindWindowW('Shell_TrayWnd', None)
    if hwnd:
        rect = RECT()
        ctypes.windll.user32.GetWindowRect(hwnd, ctypes.byref(rect))
        taskbar_top = rect.top
        # Anchor 12px from left, 6px above taskbar top edge
        self.move(12, taskbar_top - self.height() - 6)
```

B. Single-Instance Socket Lock (`main.py`)

```
import socket, sys

def enforce_single_instance():
    lock_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    try:
        lock_socket.bind(('127.0.0.1', 54321))
        return lock_socket # Retain socket lock for lifetime of process
    except OSError:
        print('Auraly is already running. Exiting duplicate process.')
        sys.exit(0)
```

7. Installation, PyInstaller Compilation & Troubleshooting

A. Deploying Pre-Built Releases

1. Download ``Auraly-Portable.zip`` from the official GitHub Releases page.
2. Unzip ``Auraly-Portable.zip`` to any folder (e.g., `C:\Program Files\Auraly`).
3. Double-click ``Auraly.exe`` inside the extracted folder.
4. (Optional) Run `python create_shortcuts.py` to add Auraly to Desktop & Windows Startup.

B. Building Standalone Executables via PyInstaller

```
# Build single executable file (.exe)
python -m PyInstaller --noconfirm --onefile --windowed --icon=app_icon.ico --add-data "app_icon.ico;."
--add-data "app_icon.png;." main.py

# Build portable directory zip release (.zip)
python -m PyInstaller --noconfirm --onedir --windowed --icon=app_icon.ico --name=Auraly --add-data
"app_icon.ico;." --add-data "app_icon.png;." main.py
```

C. Windows Defender / SmartScreen False Positives & Troubleshooting

Symptom / Error	Technical Cause	Resolution Steps
Windows Defender Block: 'File contains a virus'	PyInstaller single-file binaries extract temp files to <code>`AppData\Local\Temp`</code> . Antivirus heuristic engines flag this packer behavior as a false positive.	Use <code>`Auraly-Portable.zip`</code> (onedir build) which does not use runtime temp extraction, or go to Windows Security -> Protection History -> Allow on device .
SmartScreen Warning: 'Unknown Publisher'	Application lacks a commercial digital Code Signing Certificate (\$300+/year).	Click More info -> Run anyway in the Windows SmartScreen blue pop-up.
No Lyrics Displayed	Media player not registered with Windows GSMTTC or track has no entry on LRCLIB.	Ensure playback is active on Spotify/Chrome/Apple Music. Verify internet access for LRCLIB REST calls.

8. Future Development Roadmap

- **Custom Styling Engine:** Graphical UI picker for font family, font size, text shadow, and color gradients.
- **Real-Time Multi-Language Translation:** Dual-line rendering showing original lyrics alongside instant translation.
- **System Tray Settings Panel:** Tray icon with quick toggles for window opacity, startup launch, and manual sync offset adjustment.
- **Offline Lyric Cache:** SQLite/JSON local database caching parsed LRC files for offline playback.